31-10-2026

Agenda:

OOP

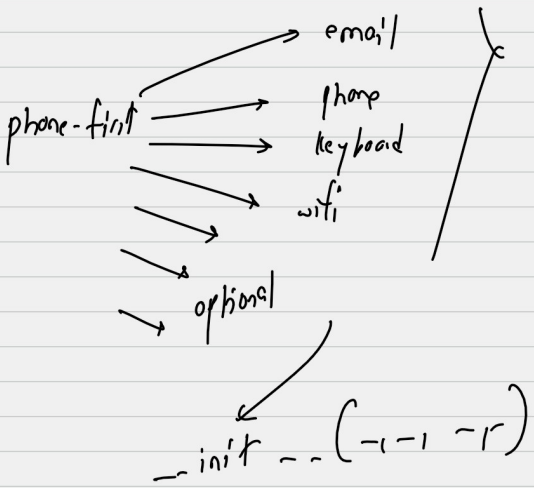
Threads & process

Components of OOP

Syntax:

1) Define a class

```
class Demo:
    pass
```




```
class Phone:

    def __init__(self, name, mail, country, wifi_name, language):
        print(language)
```

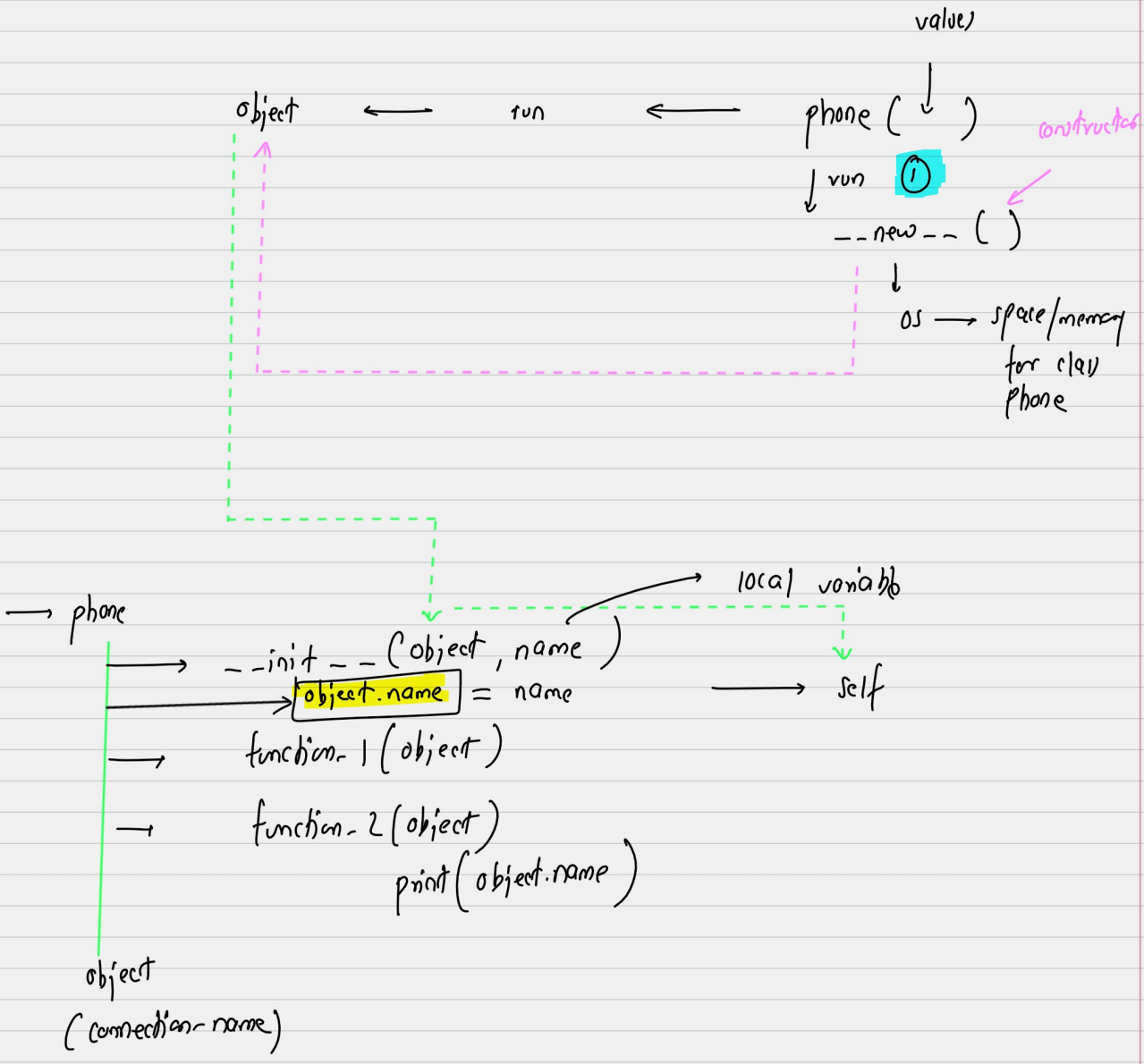
Phone( )

Phone(A, B, C, D, E)

```
class Phone:

    def __init__(self, name, mail, country, wifi_name, language):
        print(language)

    def download_from_app_store(self, app_name):
        print(f"Downloading app name for the user ({name}): {app_name}")
```




variable inside a class → instance variable

function inside a class → method

# 4 pillar of OOP

↓ imp , optional

→ inheritance , Abstract (optional)
→ polymorphism, Encapsulation (~~secret~~)

Encapsulation
Inheritance
Abstraction
polymorphism

→ standard
→ Maintain
→ Scale

## Inheritance

parent [x, y, z]
↓ train
child [x, y, z, A, B]

class parent :

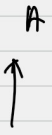
class child :

class child :

## Single-level

class A :

class B (A) :

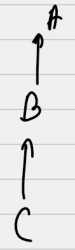



## Multi-level :

class A :

class B (A) :

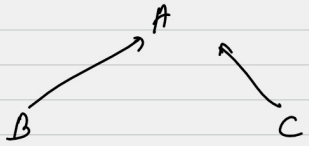
class C (B)




## Hierarchical:

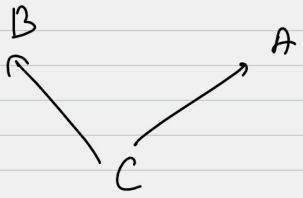
class B(A)

class C(A)



Multiple (not in java)

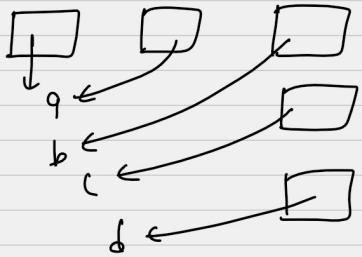
class C (A, B)




Dog IS A ANIMAL → inheritance



Tesla HAS A Engine → composition




Encapsulation:

- Java / C++ ( Data Hiding / protection )
- Python ( code readability / maintainability )
  - visual
  - naming convention →

Scope → enforced → naming convention

self.name , self.age → public → inside / sub class / outside

self._name , self._age → protected
- warning
- internal computation
- internal implementation detail
- do not touch outside of class unless you know what you are doing

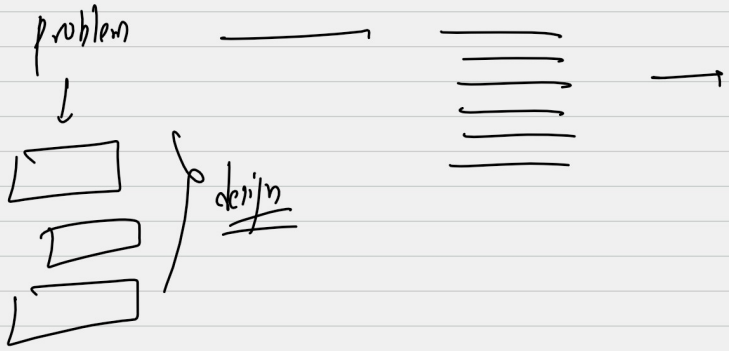
self.__name  self.__age → private → strictly internal call

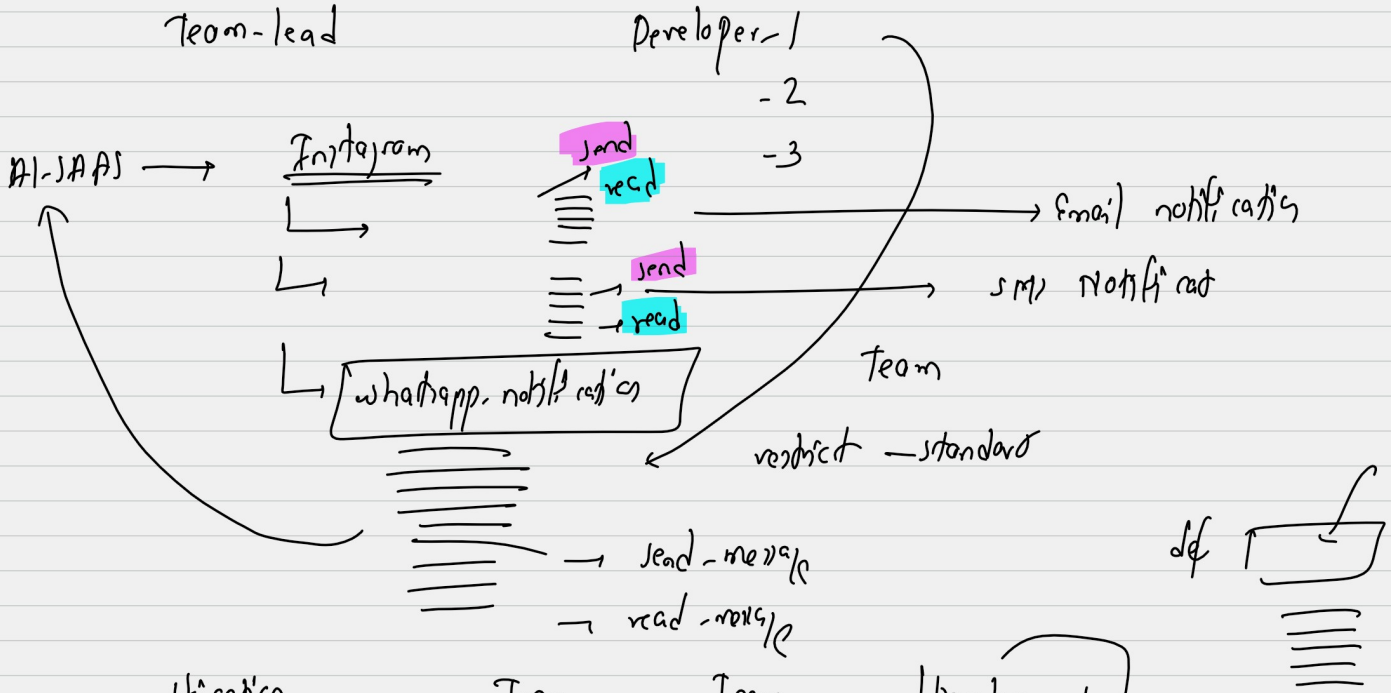
Name Mangling

self.__name # private

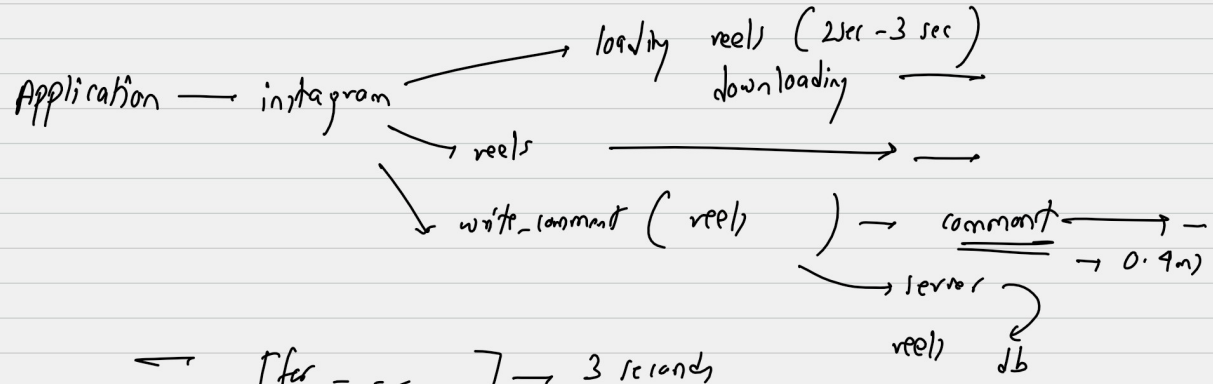
_Car__name

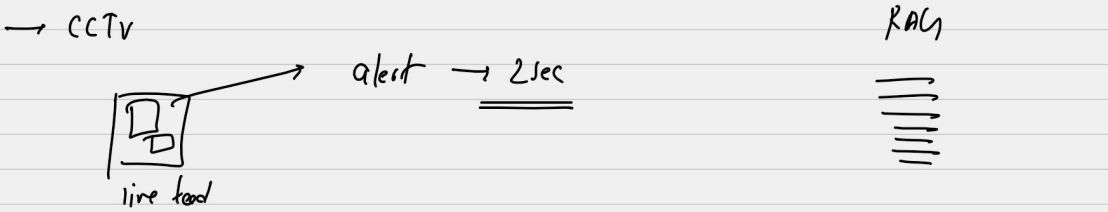
_Car__



# Real-world




CPU → core (physical) → process

12 → 24

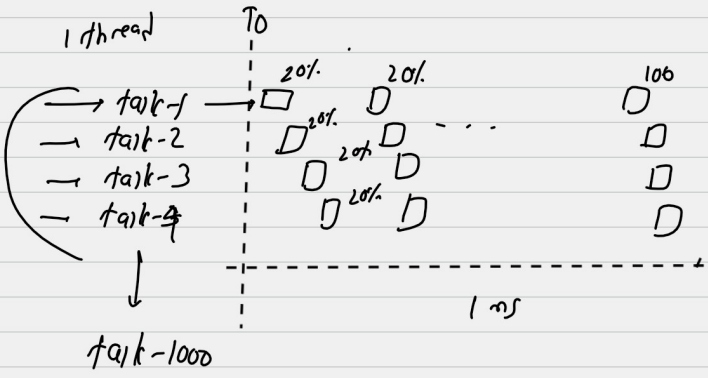
core → 1 or 2 thread
→ task
→ task

CPU
↓
12 core
↓
split-ms 24 (task)
↓
1000

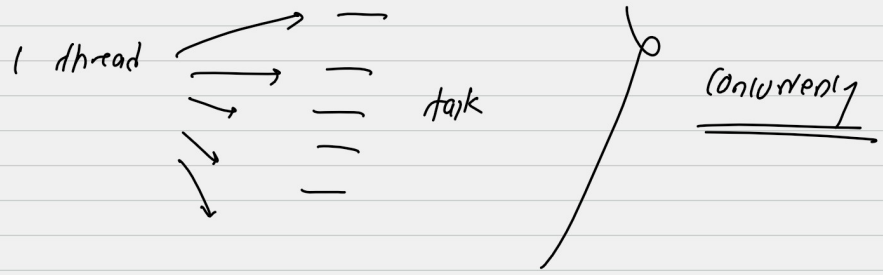
9h2

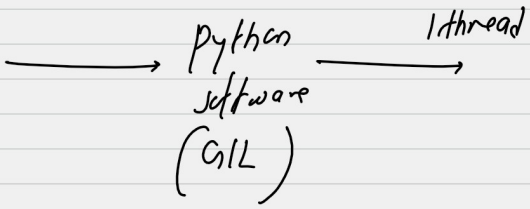
12 → 2 thread / core → 24 — virtual thread
→ 7149 thread

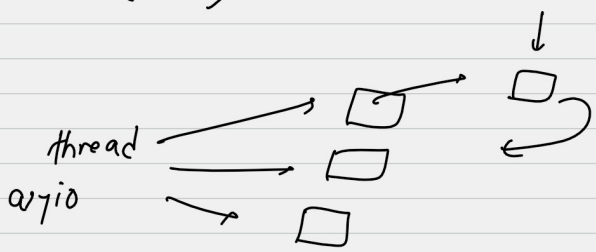
1 thread




1 to 2 →
1-2 →
1 x 2 →




thread →

thread → share memory

thread →

-

parallelism

Python

process

code → core core core

separate memory